

The Brief, and Objective 1.1: Ask Before You Draw

Chapter 0 and Chapter 1 of the Control Tower Brief. What Domain 1 is, what Meera asked for, and why nobody writes a prompt yet.

What Domain 1 is

Domain 1 is Solution Design and Architecture. It carries 17 percent of the exam, about 11 of the 63 questions. Where Domain 3 asked how the pieces connect, Domain 1 asks a question that comes before any piece exists: what should we build at all, and how big should it be?

Objective	In plain words	Weight in the brief
1.1 Business problem to Claude solution	Translate a vague ask into a workload, a constraint and a measure of better	6 of 40 questions
1.2 End-to-end architecture	Input, processing, output, and the feedback loop nobody draws	6 of 40
1.3 Pattern selection	Workflow, agent, or plain augmented LLM. Your 50 percent row from Foundations	8 of 40, soft
1.4 Multi-agent orchestration	When one agent is not enough, and who tells whom what	6 of 40
1.5 Decomposition	Fixed steps versus subtasks discovered as results come back. Your 75 percent row	8 of 40, soft
1.6 Value pillars and SLAs	Efficiency, cost, productivity, performance, transformation, each with a number	6 of 40

The brief marks 1.3 and 1.5 as soft spots because they scored 50 and 75 percent on the Foundations mock. They get the most pages, the hardest exhibits, and two extra questions each in the Blueprint Book.

The bridge before the blueprint

The picture. A city hires an engineer and says: we need to cross the river. The amateur starts drawing a suspension bridge, because suspension bridges are magnificent. The engineer first asks how many cross per day, carrying what, in what weather, and what happens when the bridge is closed. Sometimes the answer is a ferry. Sometimes a tunnel. Occasionally it really is the suspension bridge.

Domain 1 is the discipline of asking before drawing. The exam fails the candidate who draws the magnificent thing when the ferry met the constraint. Anthropic's own engineering guidance opens with the same idea: find the simplest solution possible, and add complexity only when it demonstrably improves outcomes.

Meera's ask

Monday morning, Pune. Meera Nair, Head of Operations, calls Arjun Bose into the glass room beside the operations floor. Below them, 40 dispatchers handle about 40,000 shipments a day. Exceptions arrive by email, portal ticket and phone: delays, damage, customs holds, refund demands. Meera's ask is one sentence: *I want an AI layer over this desk. The board approved budget on Friday. Tell me what we build.*

Somewhere in the pile is consignment KF-2481, Padma Deshmukh's shipment, three days late. It is the same consignment that ran through every Domain 3 experiment. Here it becomes the running example of Domain 1 as well, which is useful: you already know its facts.

Arjun's job this week is not to write a prompt. It is to answer four questions in order, and those four questions are exactly what Domain 1 grades:

#	Arjun must decide	Objective	Part of this explainer
1	What is the actual problem, and would an AI solution beat the current desk on a pillar Meera can defend to the board?	1.1, 1.6	Parts 1 and 7
2	Which shape of system: a workflow, an agent, or just an augmented LLM?	1.3	Parts 2, 3, 4
3	How does the work decompose: fixed steps, or subtasks discovered as tool results come back?	1.5	Part 5
4	If several agents, who orchestrates whom, and where do humans sit in the loop?	1.4, 1.2	Parts 6 and 7

Trap: the magnificent bridge. The exam's favourite Domain 1 wrong answer is the most capable architecture in the option list. A stem that says the steps are the same every time, or the desk must answer in two seconds, has already picked a simple workflow. Choosing the multi-agent system anyway is over-engineering, and it appears in at least a quarter of Domain 1 items. But beware the reflex in reverse: when the stem says the subtasks cannot be predicted, the agent is the right answer, and refusing it is a reversed trade-off.

LAW 1 · The constraint picks the pattern, not the ambition. Read the stem for the constraint: the known path, the latency budget, the unpredictable subtasks, the regulated data. Let it choose. An option more capable than the constraint demands is not better. On this exam it is wrong.

Objective 1.1: the doctor, not the pharmacist

The picture. Walk into a pharmacy and ask for the strongest medicine you have. A good pharmacist refuses: strongest against what? A doctor starts from the symptom, the history, the allergy list, and only then reaches for the shelf. Objective 1.1 is being the doctor. The business problem is the patient. Claude is the shelf, not the prescription.

Translating Meera's ask means rewriting it until it names three things: a **workload**, a **constraint**, and a **measure of better**. Watch the sentence sharpen:

Version	The ask	What is still missing
Meera's	An AI layer over the desk.	Everything. This is ambition, not a problem.
Better	Reduce time-to-resolution on shipment exceptions.	Volume, constraint, target.
Exam-grade	Triage about 1,200 exceptions a day into 6 categories with a draft reply, first touch under 5 minutes, refunds above Rs 10,000 always reaching a human.	Nothing. Pattern selection can begin.

Read the exam-grade version again and notice that it already whispers its own architecture. *Triage into known categories* is a router. *Draft reply* is a chained step. *Refunds above a threshold to a human* is a gate. A well-translated problem makes objective 1.3 almost mechanical. That is why 1.1 stems reward the option that clarifies before building.

Where Claude fits, and where it should not

Signal in the stem	Reading	Family if ignored
Judgement over unstructured text at volume: emails, tickets, documents	Claude-shaped work	
A deterministic rule already covers it: if carrier is X, reroute to Y	Plain code. An LLM adds cost and new failure modes	OE over-engineered
Ambiguity a human currently resolves by asking a colleague	Claude drafts, a human confirms. A gate, not a replacement	HL human loop skipped
Real-time inside a hard control loop, like sortation machinery	Not an LLM's seat. Latency and determinism rule it out	CM constraint missed

Everyday example for the second row: a shop already has a sign saying parcels over 30 kilograms go to the freight desk. You do not hire a consultant to decide, parcel by parcel, whether each one is heavy. The rule exists. Writing it as code costs nothing and never hallucinates.

The tell

1.1 stems hide the decision in the **verb**. *Classify into six known categories* is a different problem from *investigate and decide*. The first is a workflow. The second smells of an agent. Underline the verb before you look at a single option.

How this connects to Domain 3

Everything you ran in the lab sits downstream of this chapter. The router in E6 is the triage step named here. The gate in E3 is the human gate on refunds. The SOP retrieval in E4 and E5 is how the draft reply gets its facts. Domain 3 built the rooms. Domain 1 decides whether the building should exist and how many floors it needs.

The one thing to remember. Translate the ask into a workload, a constraint and a measure of better before choosing anything. The constraint picks the pattern. The verb in the stem tells you which pattern it picked.